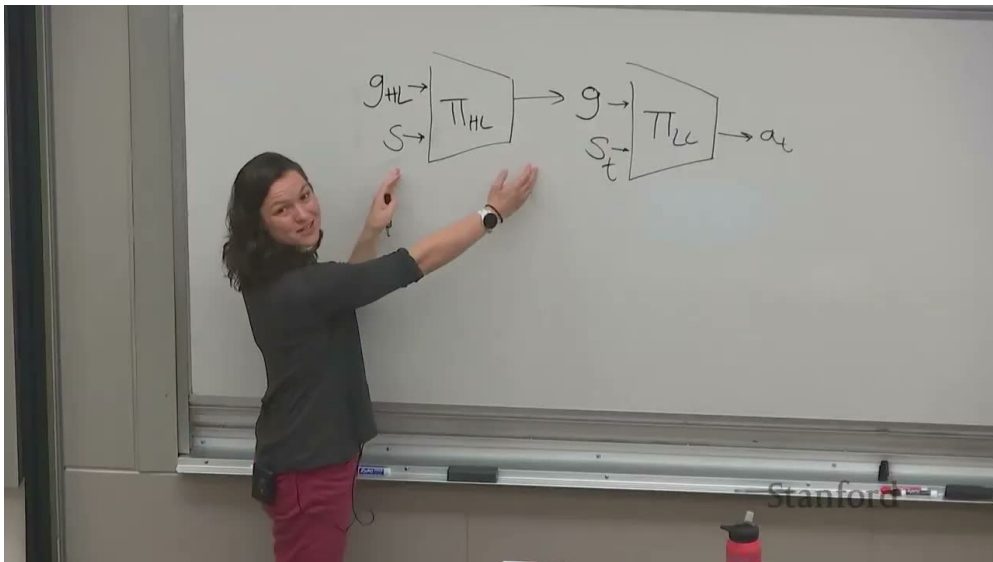


课程笔记

CS224R 第 15 讲: Hierarchy in Imitation and Reinforcement Learning

Codex 基于 Stanford CS224R 2025 春季课程视频、字幕与官方 slides 重写

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025 年 12 月 08 日

视频时长: 1:09:32

视频链接: <https://www.youtube.com/watch?v=iKWYLSVAtfM>

目录

1 长时程任务为什么难，层级为什么自然	2
1.1 本章小结	3
2 层级方法的收益与边界	3
2.1 本章小结	4
3 三大设计选择：目标表示、监督与切换时机	4
3.1 目标应该怎么表示	4
3.2 每一层应该如何监督	5
3.3 什么时候从一个子目标切到下一个	5
3.4 本章小结	6
4 分层模仿学习：语言与图像子目标	6
4.1 本章小结	8
5 分层强化学习与无监督技能发现	8
5.1 本章小结	9
6 总结与延伸	9

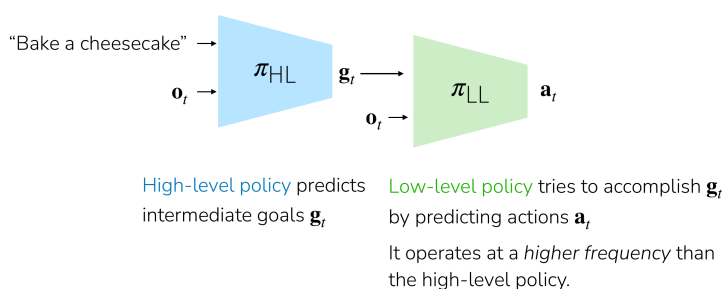
1 长时程任务为什么难，层级为什么自然

这讲一开始就把问题讲得很直接：做 focaccia、修一个导致 loss 爆炸的 bug、开车去 Yosemite、给八页报告提反馈，这些都属于 long-horizon task。它们难，不是因为单步动作特别复杂，而是因为任务持续时间长、访问状态多、错误机会多，而且很容易“卡住不前”。

如果只用一个平坦策略从原始观测直接输出动作，那么它必须在每个时刻同时考虑“当前局部动作怎么做”和“整项长期任务下一步该推进到哪里”。这会把规划、执行、恢复、共享知识等问题全部压进一个策略里，负担极重。

The main idea

Can we break up hard tasks into easier subtasks?



Many names for $\mathbf{g}_t$: subgoals, subtasks, skills, options, high-level actions

Could have more than 2 levels of hierarchy!

7

图 1: 层级策略的基本结构：高层策略提出中间目标，低层策略负责在较高频率下把目标转化为动作。

课程给出的基本答案是 hierarchy：把大任务拆成若干更容易的中间目标，再让不同层次的策略分工合作。高层策略负责决定“下一段该做什么”，低层策略负责“如何在当前控制频率下把这件事做成”。

这个想法可以写成非常简洁的两层形式：

$$g_t \sim \pi_{\text{HL}}(\cdot \mid o_t, g^{\text{task}}), \quad a_t \sim \pi_{\text{LL}}(\cdot \mid o_t, g_t).$$

这里：

- o_t 是当前观察；
- g^{task} 是最终任务目标或任务描述；
- g_t 是高层策略在时刻 t 给出的中间目标、技能或 option；
- π_{HL} 是高层策略；
- π_{LL} 是低层策略；
- a_t 是由低层策略输出的实际动作。

讲者特别强调， g_t 可以有很多名字：subgoal、subtask、skill、option、high-level action。重要的不是命名，而是中间表示是否足以把长时程问题拆成更可解的小块。

这讲的中心命题

层级方法并不神秘，它试图解决的就是一个朴素问题：如果完整任务太长太难，能否把“选下一步做什么”和“把当前这步做好”分开建模。

1.1 本章小结

长时程任务的难点来自状态空间扩张、易出错和易陷入停滞。层级方法的自然性在于，它把长期规划与短期执行拆分开，让每一层专注更合适的时间尺度与抽象层次。

2 层级方法的收益与边界

课程给出四个主要收益。第一，层级可以为低层策略提供更直接的中间监督，而不是只靠最终任务成功信号。第二，相近子任务之间更容易共享知识，因为低层是一个条件在目标上的多任务策略。第三，在 RL 中，高层目标还能为探索提供结构。第四，在工程实现上，高层与低层可以运行在不同频率，从而降低计算成本。

Why may hierarchy help?

1. Supervision signal on how to complete the task
2. More direct knowledge sharing across similar subtasks
3. In RL: structured exploration in higher-level space
4. Practical challenge: Address speed requirements by running policies at different frequencies

9

图 2: 课程总结层级方法的四类收益：中间监督、跨子任务知识共享、结构化探索，以及不同频率下的计算优势。

这里最值得注意的是“不同频率”这个点。低层策略常常需要贴近控制频率，例如机器人 20Hz 或更高；高层策略则只需在完成一段子任务后重新规划。这样一来，昂贵的高层推理不必在每个时间步重复执行。

但课程也没有把 hierarchy 神化。它专门拿“层级策略”“平坦策略”“chain-of-thought 风格单策略”三种做法并排比较，提醒我们：层级并不一定意味着必须有两个完全独立的模型。有时你也可以在单个模型里显式生成中间步骤，只是这种做法在高频控制场景中可能计算代价太高。

Do we *really* need multiple policies?

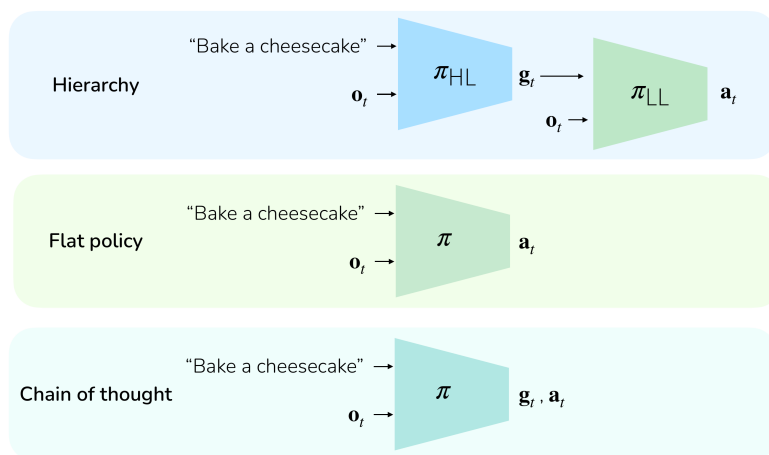


图 3: Hierarchy、flat policy 与 chain-of-thought 的对比提醒我们：多层结构是一种设计选择，而不是唯一答案。

课程在这里给出的判断相当稳妥：当前还没有足够结论证明哪一类结构在所有场景下都压倒性更优。是否显式拆层，取决于任务类型、监督形态、控制频率和计算预算。

为什么层级常在机器人里更有吸引力

在高频连续控制里，如果每一步都让模型重新做长链推理，计算成本和延迟都会迅速变得不可接受。层级结构天然适合把“慢思考”与“快控制”分开。

2.1 本章小结

层级方法的价值并不只在“更像人类分步骤思考”，更在于它为监督、知识共享、探索和算力分配提供了结构。但它也不是万灵药，flat policy 或单模型显式推理在某些任务上同样可能成立。

3 三大设计选择：目标表示、监督与切换时机

这一讲中段最重要的部分，是把 hierarchy 设计问题拆成三个具体问题。

3.1 目标应该怎么表示

课程先让同学讨论不同场景下的中间目标表示。例如，骑车去校园不同地点可以用 GPS 坐标，做菜可以用语言描述，视觉任务则可以用图像状态。讲者随后给出三个标准：

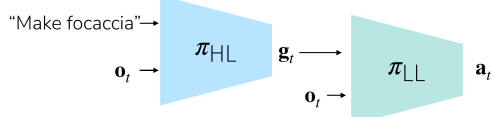
- 表达能力强：能覆盖足够多的低层行为；
- 结构化：相近目标在表示空间里也应相近；
- 抽象层次合适：既不能细到高层失去意义，也不能粗到低层根本做不到。

这说明“最佳目标表示往往是 domain-specific 的”。课程并没有鼓吹一种统一表征，而是把重点放在“这个表示是否有助于分解任务”。

3.2 每一层应该如何监督

如果把所有东西都 end-to-end 地学成一个 latent goal 表示，会发生什么？课程的答案是：很可能又退化回 flat policy，因为中间表示没有被明确约束出有用的结构。为了真正得到 hierarchy 的好处，必须认真考虑高层和低层分别接收什么监督。

How to supervise each level?



Why not train everything end-to-end, with latent goal representation?

Result: a flat policy

Key: Think carefully about where the benefits are coming from!

Generally good to do in any kind of research!!

16

图 4: 层级系统的监督问题：高层学会选对中间目标，低层学会把当前目标完成。

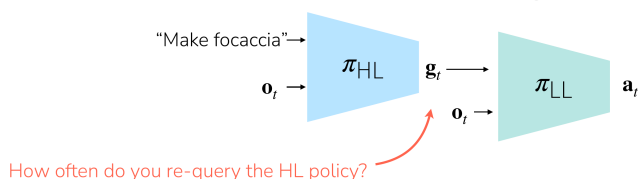
课程建议的一般思路是：高层围绕长期任务回报或高层目标接受监督，低层围绕 goal-reaching 或子任务成功接受监督。这样才能让低层真正学会“完成子任务的共性能力”，而不是把整件事重新压回一个平坦模型里。

3.3 什么时候从一个子目标切到下一个

这部分非常工程化，也很重要。课程给了两种基本做法：

- 基于完成判断切换：当系统认为当前子目标已经完成时，才请求高层重新规划。
- 基于固定时间间隔切换：例如低层 50Hz，高层每秒更新一次，那么每 50 步重采样一个子目标。

When to move on from one subgoal to the next?



- Rolling out the hierarchical policy:
0. Observe initial observation $\mathbf{o}_1$ at $t = 1$
 1. Plan goal $\mathbf{g}_t$ according to $\pi_{HL}(\cdot | \mathbf{o}_t, \text{prompt})$
 2. Until new goal is selected:
 - a. Execute predicted action $\mathbf{a}_t$ according to $\pi_{LL}(\cdot | \mathbf{o}_t, \mathbf{g}_t)$
 - b. $t \leftarrow t + 1$, observe new observation $\mathbf{o}_t$

20

图 5: 子目标切换既可以基于“是否完成”的判别器，也可以基于固定时间间隔。两者都有利弊。

前者在理论上更理想，因为高层只在真正需要时思考；但它依赖一个可靠的完成判别器，而且必须处理“中途犯错需要回退”的情况。后者更简单，但可能会在不该切换时切换，或者在该重规划时反应太慢。

课程给出的务实建议

在真实系统里，层级最难的往往不是网络结构，而是界面设计：目标表示是否可操作，监督是否分层合理，切换逻辑是否稳健。

3.4 本章小结

目标表示、监督方式和切换时机，构成了 hierarchy 的三根支柱。没有这三项设计，所谓层级结构很容易名义上分层、实际上退化回平坦策略。

4 分层模仿学习：语言与图像子目标

讲者随后给出若干近年的机器人系统例子，先从 hierarchical imitation learning 开始。第一类是语言子目标：把长时程 demonstration 切成若干 segment，每段配一个语言命令，作为高层策略的监督。

Hierarchical imitation with language subgoals

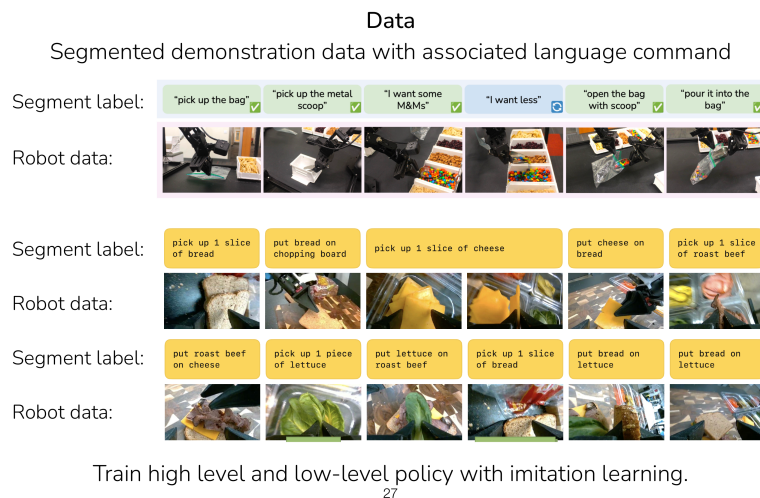


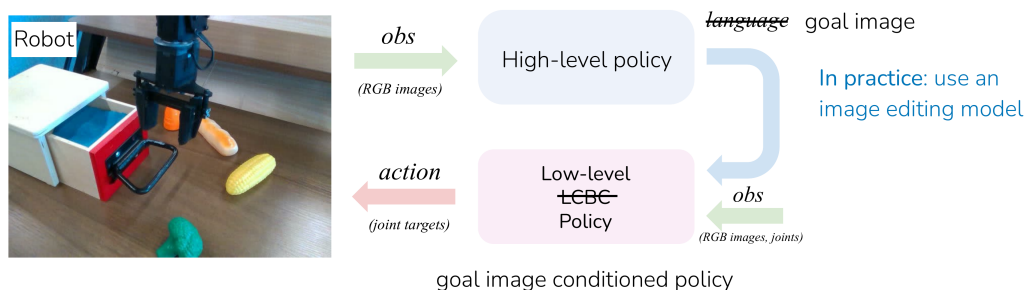
图 6: 语言子目标训练数据的典型形态: 一条完整 demonstration 被切成多个带语言标签的片段。

在这种设置下, 低层策略看到当前观测和语言子目标, 输出细粒度动作; 高层策略则根据当前观测生成下一条语言指令。课程进一步说明, 这类系统可以配合 DAgger 使用, 而且高层 DAgger 甚至可以在语言空间里进行。换句话说, 人类不必直接纠正每个关节动作, 只需要说“下一步应该去拿袋子”“不要把东西倒到袋子外面”即可。

这使 hierarchy 不只是一种结构分解, 也是一种更便于人类提供监督的接口设计。讲者引用的机器人例子里, 高层大约在 1Hz 左右规划, 而低层以 50Hz 左右输出控制命令, 正好体现了不同时间尺度的分工。

第二类是图像子目标。这里高层不再输出语言, 而是输出中间视觉目标, 例如“目标状态应该长什么样”。这对需要精细视觉控制的机器人任务尤其自然, 因为低层能直接把当前观察和目标图像对齐。

Hierarchical imitation with image subgoals



Benefits: No need for segmented videos with language annotations.
High-level policy can incorporate unlabeled video data!

图 7: 图像子目标把中间目标表示成视觉状态, 从而让高层规划与低层控制共享同一视觉空间。

课程在这里还提到一个重要研究问题：训练高层时是否应加入人类视频，而不只用机器人数据？这反映出 hierarchy 不只是“多一层 policy”，还涉及多源数据如何进入不同层级。

4.1 本章小结

分层模仿学习展示了 hierarchy 的一个巨大优势：中间目标既可以是语言，也可以是图像，从而把高层规划暴露成更自然、更可标注、更可纠正的接口。

5 分层强化学习与无监督技能发现

最后一部分把 hierarchy 放回 RL 语境。课程展示的典型做法是：低层是 goal-conditioned policy，以达到目标状态为奖励；高层则输出目标状态，把这些状态看作高层动作。

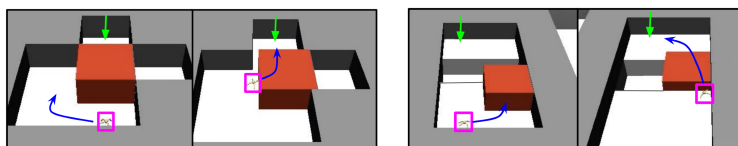
Hierarchical reinforcement learning

State reaching goals (e.g. relative target position of agent & objects)

Low-level: Goal-conditioned policy, with goal-reaching reward

High-level: Train policy to output goal states (these are the HL actions)

Use hindsight relabeling on HL actions, using off-policy algorithm



Nachum, Gu, Lee, Levine. Data-Efficient Hierarchical Reinforcement Learning. NeurIPS '18.

Language goals (for simple semantic tasks)

Low-level: Language-conditioned policy, with hindsight *language* relabeling

High-level: Train policy to output language

Jiang, Gu, Murphy, Finn. Language as an Abstraction for Hierarchical Deep Reinforcement Learning. NeurIPS '19.

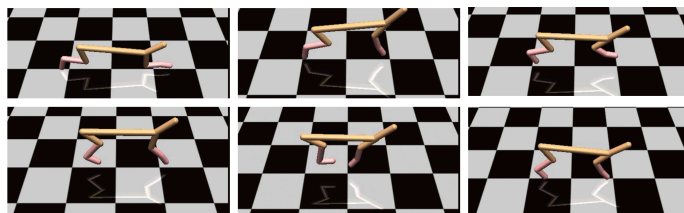


图 8: 分层 RL 的标准结构：低层学会达到目标状态，高层在更抽象的目标空间里做决策。

这类方法常配合 hindsight relabeling 使用。因为高层给出的目标未必总是实现，但事后可以根据实际到达的状态重标注目标，从而提高样本效率。课程还提到另一条线：把高层目标做成语言，而不是状态，这样系统就能在更抽象、更可解释的空间里规划。

更进一步，还有一个非常活跃的研究方向是不依赖外部奖励函数，直接学习一组多样化技能。课程把它作为 hierarchy 的相关延伸：如果低层能先自发学出一组覆盖广、彼此可区分的技能，那么高层后续就可以在技能空间中做组合与规划。

Another active research area:
Can you *discover* a set of diverse skills without supervision?



Eysenbach, Gupta, Ibarz, Levine. Diversity is all you need: Learning diverse skills without a reward function. ICLR 2019

44

图 9: 无监督技能发现把“先学一组可复用 primitive”作为 hierarchy 的前置能力。

这条线的意义在于，它把“层级结构的有用性”往前再推一步：不仅高层要学会组合低层，低层本身也可能先通过无监督目标学出足够丰富的行为库。

5.1 本章小结

分层 RL 说明 hierarchy 不只是模仿学习里的监督接口设计，也能成为 RL 中的探索与信用分配工具；无监督技能发现则进一步把低层技能库本身变成研究对象。

6 总结与延伸

这一讲把 hierarchy 从动机、设计到实例完整讲清楚了。其逻辑链条非常清晰：

- 长时程任务难在任务链太长、状态太多、错误与停滞太容易发生；
- 层级方法通过中间目标把长期规划与短期执行解耦；
- 真正决定方法是否有效的，是目标表示、监督分层和切换时机这三项设计；
- 在机器人和 RL 中，语言、图像、目标状态乃至无监督技能，都可以充当不同层次的接口。

这讲也给出一个重要研究态度：不要把 hierarchy 理解成“必须有两个 policy 的固定套路”。它更像是一类设计原则，强调利用不同抽象层次、不同时间尺度和不同监督接口来组织决策过程。因此，它既可以体现在显式高低层策略里，也可能体现在带中间步骤的单模型推理里。

对后续学习而言，这讲最值得记住的是 hierarchy 的工程含义。很多系统真正卡住的地方，并不是“模型不够大”，而是没有把任务分解成合适的中间接口。只要长期任务仍然存在，hierarchy 就会继续是 imitation learning、robotics 和 reinforcement learning 里的核心主题。